

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении практической работы №2
по дисциплине «Технологии разработки программного
обеспечения»
«Разработка клиент-серверного приложения»

Выполнила студентка группы

№5130201/30101

Проверил

Лаишевкина А.М. _____

Силиненко А.В. _____

«_____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	4
1 Спецификация требований	5
1.1 Общее описание задачи	5
1.2 Описание интерфейса	5
1.2.1 Входные данные	5
1.2.2 Выходные данные	5
1.2.3 Информационные сообщения	6
1.2.4 Хранимые данные	7
1.3 Функциональные требования	7
1.3.1 Ввод данных	7
1.3.2 Проверка корректности данных	7
1.3.3 Вычисление минимального пути	8
1.3.4 Сетевое взаимодействие	8
1.3.5 Вывод результатов и завершение работы	8
1.4 Требования к тестированию	9
2 Проектирование	10
2.1 Перечень модулей	10
2.1.1 Модули клиентской части	10
2.1.2 Модули серверной части	11
2.2 Алгоритмы работы приложения при функционировании по протоколу UDP	11
2.2.1 Алгоритмы работы клиентской части	11
2.2.2 Алгоритмы работы серверной части	12
2.3 Форматы структур данных, передаваемые между клиентом и сервером	12
3 Разработка приложения	15
3.1 Среда разработки	15
3.2 Используемый компилятор	15
3.3 Основные технологии, используемые в приложении	15
3.3.1 Механизм межпроцессного взаимодействия	15
3.3.2 Механизм мультиплексирования ввода и вывода	16
3.4 Примеры запуска и работы программы	16
4 Тестирование	18
4.1 Средства разработки тестирующего приложения	18
4.2 Протоколы тестирования	18

Заключение	20
Приложение А. Исходный код клиентской части	21
Приложение Б. Исходный код серверной части	29
Приложение В. Исходный код тестирующего приложения (bash-скрипт)	36
Приложение Г. Исходный код тестирующего приложения (Expect-скрипт)	38
Приложение Д. Протоколы тестирования	42

Введение

Данная работа посвящена разработке клиент-серверного приложения. Такая архитектура программы подразумевает, что одна программа (сервер) предоставляет ресурсы или вычислительные услуги, а другая (клиент) запрашивает и использует их через сетевое соединение. В данной работе реализовано приложение, решающее задачу поиска кратчайшего пути в графе: клиент формирует запрос с описанием графа и отправляет его серверу, а сервер выполняет вычисления и возвращает результат. Приложение поддерживает работу как по протоколу TCP, так и по UDP, а также включает автоматизированную систему тестирования и обработку одновременных подключений с помощью мультиплексирования.

Актуальность данной работы обусловлена тем, что клиент-серверная архитектура остаётся основой большинства современных систем — от веб-сервисов и облачных приложений до баз данных и игровых серверов. Поэтому получение практических навыков проектирования и разработки приложений с такой архитектурой является необходимостью для разработчика. Задача поиска минимальных путей также широко используется в современных информационных системах, что повышает актуальность работы.

Цель: разработка клиент-серверного приложения, вычисляющего длину минимального пути в неориентированном графе.

Задачи:

1. Получение базовых знаний по разработке приложений в ОС Linux.
2. Получение знаний и практических навыков реализации межпроцессных взаимодействий с помощью механизма сокетов.
3. Получение знаний по работе с графами.
4. Разработка клиент-серверного приложения.
5. Тестирование клиент-серверного приложения.

1 Спецификация требований

1.1 Общее описание задачи

Назначение программы: Клиент-серверное приложение для вычисления длины минимального пути между вершинами в неориентированном графе.

Архитектура: x86.

Операционная система: Linux.

Язык реализации: C/C++.

Сетевые протоколы: TCP и UDP.

1.2 Описание интерфейса

1.2.1 Входные данные

Входные данные сервера

- Тип подключения: TCP/UDP.
- Векторы смежности графов в виде последовательностей пар: номер соседней вершины и вес ребра к ней.

Входные данные клиента

- Тип подключения: TCP/UDP.
- IP-адрес сервера и порт серверной части.
- Строки, которые являются векторами смежности графа и представляют собой последовательность пар: номер соседней вершины и вес ребра к ней, – вводимые с клавиатуры или читаемые из файла.
- Результат вычисления сервером минимального пути между вершинами и список вершин минимального пути.

1.2.2 Выходные данные

Выходные данные сервера

Вычисленное значение минимального пути для заданного графа и список вершин минимального пути или сообщение об ошибке.

Выходные данные клиента

- Структура данных, содержащая описание графа.
- Вывод результата на экран.

1.2.3 Информационные сообщения

Клиентская часть

- Приглашение к вводу данных графа.
- Сообщения о состоянии подключения к серверу.
- "Подключение к серверу установлено."
- "Потеряна связь с сервером"(для UDP после 3 неудачных попыток)
- Сообщение со списком команд для взаимодействия с программой (– help).

Серверная часть

- "Сервер запущен на порту X."
- "Ожидание подключений..."
- "Новое TCP подключение от IP:PORT"
- "Получен UDP-запрос от клиента."
- "Получен запрос от клиента."
- "Сервер завершает работу..."(по сигналу прерывания)

Сообщения об ошибках

- "Ошибка: должно быть не менее 6 вершин!"
- "Ошибка: должно быть не менее 6 рёбер!"
- "Ошибка: должно быть не более 20 вершин!"
- "Ошибка: отрицательный вес рёбер!"
- "Ошибка: граф несвязный!"
- "Ошибка: неверные номера вершин! Должны быть от 0 до N."

- "Ошибка создания сокета."
- "Ошибка подключения к серверу."
- "Ошибка отправки/получения данных."

1.2.4 Хранимые данные

Векторы смежности для представления графа.

Временные структуры данных для алгоритма Дейкстры.

1.3 Функциональные требования

1.3.1 Ввод данных

Клиентская часть

- Программа должна принимать параметры командной строки: IP-адрес сервера, номер порта и транспортный протокол (TCP/UDP).
- Программа должна выводить приглашающие сообщения для ввода данных графа.
- Программа должна корректно обрабатывать ввод всех необходимых параметров графа.

Серверная часть

- Сервер должен запускаться на указанном порту и ожидать подключений.
- Сервер должен одновременно обрабатывать не менее 3 клиентских подключений.

1.3.2 Проверка корректности данных

Клиентская часть

- Программа должна проверять корректность введённых списков смежности графа.
- Программа должна проверять корректность начальной и конечной вершин.
- Программа должна проверять корректность введённых весов рёбер.

Серверная часть

- Сервер должен повторно проверять корректность полученных данных графа.
- Сервер должен проверять связность графа и существование пути между вершинами.
- Сервер должен возвращать клиенту соответствующие коды ошибок при некорректных данных.

1.3.3 Вычисление минимального пути

- Сервер должен реализовывать алгоритм Дейкстры для взвешенного неориентированного графа с неотрицательными весами для поиска кратчайшего пути в графе.
- Сервер должен корректно обрабатывать неориентированные графы.
- Сервер должен возвращать длину минимального пути и список вершин минимального пути или сообщение о его отсутствии.

1.3.4 Сетевое взаимодействие

- Программа должна поддерживать работу как по TCP, так и по UDP протоколам.
- Для UDP должна быть реализована система подтверждения доставки с повторными отправлениями.
- Клиент и сервер должны корректно обрабатывать разрыв соединения.
- Программа должна работать как на одном компьютере, так и в сетевой среде.
- Программа должна поддерживать одновременную работу как минимум с 3 клиентами.

1.3.5 Вывод результатов и завершение работы

- Клиентская часть должна выводить строку «Минимальный путь =» и вычисленное значение длины минимального пути.

- Клиентская часть должна завершать работу по команде пользователя.
- Серверная часть должна работать непрерывно и завершаться только по сигналу от пользователя.
- Завершение клиентов не должно влиять на работу сервера.

1.4 Требования к тестированию

Требования к тестированию включают в себя проверку:

- работы приложения по протоколу TCP;
- работы приложения по протоколу UDP, включая работу клиента при недоступности сервера;
- ввода исходных данных с клавиатуры;
- ввода исходных данных из файла;
- работы приложения при задании графа, количество вершин которого близко к граничным значениям области допустимых значений (ОДЗ):
 - граф с 5-ю вершинами (ниже нижней границы ОДЗ);
 - граф с 6-ю вершинами (нижняя граница ОДЗ);
 - граф с 20-ю вершинами (верхняя граница ОДЗ);
 - граф с 21-й вершиной (выше верхней границы ОДЗ);
- работы приложения при задании значений количества вершин из середины ОДЗ.

2 Проектирование

2.1 Перечень модулей

2.1.1 Модули клиентской части

Основной модуль

Назначение модуля:

- управление жизненным циклом клиентского приложения;
- обработка аргументов командной строки (IP, порт, протокол);
- создание и настройка сетевого соединения;
- вызов соответствующих обработчиков ввода/вывода, передачи запроса серверу и получения результата;
- вывод информационных сообщений (приглашение к вводу данных, сообщения об ошибках) и результата вычислений.

Модуль ввода и обработки данных

Назначение модуля:

- ввод структуры данных, описывающей граф, с клавиатуры или из файла;
- проверка корректности вводимых данных;
- преобразование введенной структуры во внутреннее представление графа.

Сетевой модуль

Назначение модуля:

- установление соединения с сервером по TCP или UDP;
- отправка данных графа и параметров поиска кратчайшего пути;
- получение результата вычислений от сервера;
- обработка ошибок сетевого взаимодействия;
- для UDP-подключения: механизм повторных попыток и подтверждений при отправке, подтверждение при получении результата от сервера.

2.1.2 Модули серверной части

Основной модуль

Назначение модуля:

- инициализация сервера;
- инициализация и привязка сокета к заданному порту;
- запуск основного цикла обработки запросов;
- обработка сигналов завершения работы сервера.

Модуль обработки графа

Назначение модуля:

- проверка корректности полученных данных;
- реализация алгоритма Дейкстры для поиска кратчайшего пути;
- построение последовательности вершин минимального пути;
- формирование полного ответа клиенту, состоящего из длины минимального пути и последовательности вершин в нем.

Сетевой модуль

Назначение модуля:

- прием входящих TCP и UDP сообщений;
- для UDP: отправка подтверждения получения данных;
- отправка результата клиенту;
- обработка ошибок сетевого взаимодействия.

2.2 Алгоритмы работы приложения при функционировании по протоколу UDP

2.2.1 Алгоритмы работы клиентской части

Алгоритм работы клиентской части при использовании протокола UDP приведен на [Рис. 1](#).

2.2.2 Алгоритмы работы серверной части

Алгоритм работы серверной части при использовании протокола UDP приведен на [Рис. 2](#).

2.3 Форматы структур данных, передаваемые между клиентом и сервером

Клиент передает серверу структуру данных, содержащую:

- количество вершин графа N : `int`;
- матрицу весов рёбер размером $N \times N$: `vector<vector<int>>`;
- индекс начальной вершины: `int`;
- индекс конечной вершины: `int`.

В случае отсутствия ребра между вершинами соответствующий элемент матрицы содержит значение `INT_MAX`, обозначающее бесконечность.

Сервер передает клиенту структуру данных, содержащую:

- длину минимального пути: `int` (или `-1`, если путь не существует);
- последовательность вершин кратчайшего пути или сообщение об отсутствии пути: `vector<int>` (содержит индексы вершин) или пустой вектор.

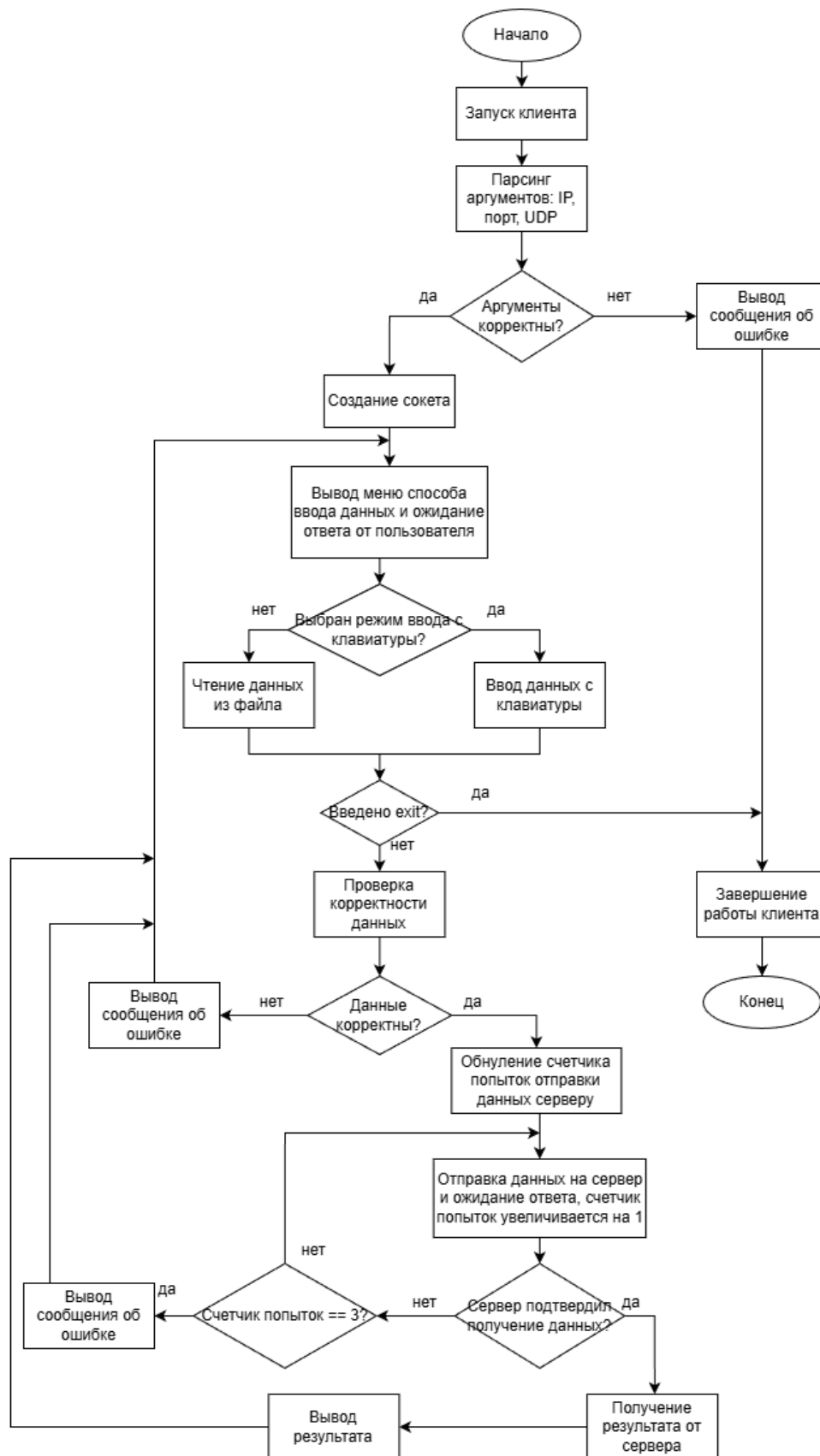


Рис. 1. Алгоритм работы клиентской части (UDP)

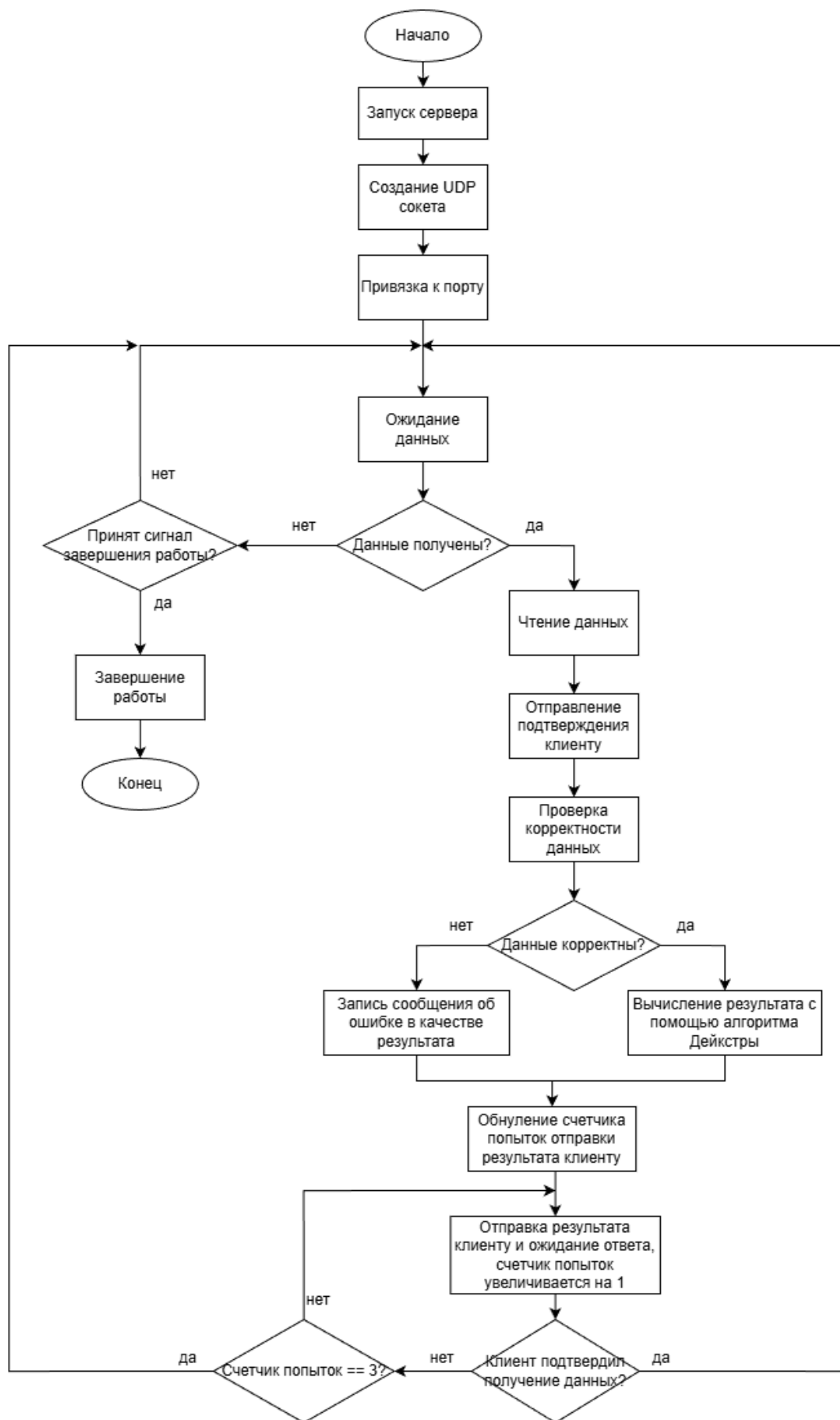


Рис. 2. Алгоритм работы серверной части (UDP)

3 Разработка приложения

3.1 Среда разработки

Разработка клиент-серверного приложения проводилась в операционной системе Linux Ubuntu 24.04.3 LTS. В качестве среды разработки использовался текстовый редактор Visual Studio Code. Сборка, запуск и отладка приложения выполнялись в стандартном терминале Ubuntu с помощью командной строки.

3.2 Используемый компилятор

Для компиляции исходного кода использовался компилятор GNU g++, входящий в состав пакета GNU Compiler Collection (GCC). Сборка проекта осуществляется с использованием утилиты make и файла сборки Makefile. В процессе сборки применяется стандарт языка C++17. В результате сборки исходных файлов на язык C++ были сформированы исполняемые файлы client и server.

3.3 Основные технологии, используемые в приложении

3.3.1 Механизм межпроцессного взаимодействия

В приложении реализован обмен данными между процессами организован с использованием TCP/IP-сокетов. На стороне сервера сначала создаётся слушающий сокет, который затем привязывается к конкретному сетевому адресу и порту, после этого сервер переходит в режим ожидания входящих подключений. На стороне клиента также создается сокет, затем клиент устанавливает соединение с сервером, после успешного установления соединения клиент может отправлять данные и ожидать ответ от сервера. После получения результата клиент завершает сеанс.

Используемые системные вызовы:

- На стороне сервера:
 - `socket()` — создаёт TCP-сокет.
 - `bind()` — привязывает сокет к IP-адресу и порту.
 - `listen()` — переводит сокет в режим ожидания входящих соединений.

- `accept()` — принимает новое клиентское соединение и создаёт новый сокет для общения с клиентом.
- `recv()` / `read()` и `send()` / `write()` — для приёма и отправки данных.
- `close()` — закрывает сокет.

- На стороне клиента:

- `socket()` — создаёт клиентский сокет.
- `connect()` — устанавливает соединение с сервером по заданному адресу и порту.
- `send()` / `write()` и `recv()` / `read()` — для передачи запроса и получения ответа.
- `close()` — закрывает соединение.

3.3.2 Механизм мультиплексирования ввода и вывода

Для обеспечения возможности одновременной работы сервера с несколькими клиентами в серверной части используется механизм мультиплексирования ввода-вывода. Мультиплексирование реализовано с помощью системного вызова `select()`. Этот механизм позволяет серверу отслеживать активность нескольких сокетов и, соответственно, обрабатывать запросы нескольких клиентов одновременно.

3.4 Примеры запуска и работы программы

Для успешной сборки приложения необходимо выполнить команду `make` в корневой директории проекта. В результате будут сформированы исполняемые файлы `server` и `client`.

Запуск серверной части осуществляется командой `./server <PORT>`. Если номер порта не указан, сервер по умолчанию запускается на порту 8080.

Запуск клиентской части выполняется командой `./client <IP> <PORT> <PROTOCOL>`, где `<IP>` — IP-адрес сервера, `<PORT>` — номер порта сервера, `<PROTOCOL>` — используемый протокол.

После запуска клиент переходит в режим ожидания ввода данных от пользователя. Пользователь может ввести описание графа вручную с клавиатуры (в формате: количество вершин, матрица весов, индексы начальной и конечной вершины) или загрузить данные из файла. После

передачи данных на сервер клиент ожидает ответ и отображает полученный результат: либо длину минимального пути и последовательность вершин, либо сообщение об отсутствии пути.

Исходный код программы приведен в приложениях [А](#) и [Б](#).

4 Тестирование

Для тестирования было написано отдельное приложение. Это помогло автоматизировать проверку корректности работы клиент-серверного приложения.

4.1 Средства разработки тестирующего приложения

Тестирующее приложение разработано с использованием языка сценариев `Expect`, расширения языка `Tcl`, предназначенного для автоматизации взаимодействия с интерактивными консольными программами.

Также для тестирования использовался `bash`-скрипт, который является текстовым файлом с командами, которые обычно вводят в терминал вручную; он позволяет автоматизировать последовательность действий в `Linux`.

Логика тестирующего приложения подразумевает, что `bash`-скрипт управляет всем процессом тестирования, а `Expect` имитирует действия человека при работе с клиентом.

4.2 Протоколы тестирования

Для запуска и координации тестов используется `Bash`-скрипт `run_all_tests.sh`, который обеспечивает:

- проверку наличия исполняемых файлов;
- запуск сервера на заданном порту;
- последовательное выполнение тестовых сценариев через `Expect`;
- обработку результатов и завершение сервера после тестирования.

Написанный сценарий `expect_graph_test.exp` автоматически:

- запускает клиентскую часть с указанным `IP`, портом и протоколом;
- передаёт тестовые данные (описание графа, индексы вершин);
- проверяет наличие ожидаемого результата или сообщения об ошибке.

Написанные тестирующие скрипты приведены в приложениях [В](#) и [Г](#). В ходе тестирования с помощью автоматических тестов проверяется:

- работа приложения по протоколу `TCP`;

- работа приложения по протоколу UDP, включая работу клиента при недоступности сервера;
- ввод исходных данных с клавиатуры;
- ввод исходных данных из файла;
- работа приложения при задании графа, количество вершин которого близко к граничным значениям области допустимых значений (ОДЗ):
 - граф с 5-ю вершинами (ниже нижней границы ОДЗ);
 - граф с 6-ю вершинами (нижняя граница ОДЗ);
 - граф с 20-ю вершинами (верхняя граница ОДЗ);
 - граф с 21-й вершиной (выше верхней границы ОДЗ);
- работа приложения при задании значений количества вершин из середины ОДЗ.

Тестирование выполняется для обоих протоколов — TCP и UDP, что гарантирует корректность сетевого взаимодействия независимо от используемого транспортного протокола.

Программа успешно прошла все тесты. Их результаты фиксируются в логах, а итоговый вывод скрипта `run_all_tests.sh` показывает общее количество пройденных и неудачных тестов. Вывод тестирующего приложения приведен в приложении [Д](#).

Заключение

В результате работы была выполнена поставленная цель – разработка клиент-серверного приложения, вычисляющего длину минимального пути в неориентированном графе.

Также были решены все поставленные задачи:

1. Получены базовые знания по разработке приложений в ОС Linux.
2. Получены знания и практические навыки реализации межпроцессных взаимодействий с помощью механизма сокетов.
3. Получены знания по работе с графами.
4. Разработано клиент-серверное приложение.
5. Протестировано клиент-серверное приложения.

Приложение А. Исходный код клиентской части

```
1 #include <arpa/inet.h>
2 #include <errno.h>
3 #include <fcntl.h>
4 #include <netinet/in.h>
5 #include <poll.h>
6 #include <sys/socket.h>
7 #include <unistd.h>
8
9 #include <algorithm>
10 #include <cstring>
11 #include <fstream>
12 #include <iostream>
13 #include <sstream>
14 #include <string>
15 #include <vector>
16
17 // удаление пробелов в начале и конце строки
18 static inline std::string trim(const std::string& s) {
19     size_t a = s.find_first_not_of(" \t\r\n");
20     if (a == std::string::npos) return "";
21     size_t b = s.find_last_not_of(" \t\r\n");
22     return s.substr(a, b-a+1);
23 }
24
25 // чтение файла
26 bool read_file(const std::string& fn, std::string& out) {
27     std::ifstream f(fn);
28     if (!f.is_open()) return false;
29     std::string line;
30     out.clear();
31     while (std::getline(f, line)) out += line + "\n";
32     return true;
33 }
34
35 // разделение текста на строки
36 std::vector<std::string> split_lines(const std::string& txt) {
37     std::vector<std::string> v;
38     std::istringstream iss(txt);
39     std::string line;
40     while (std::getline(iss, line)) {
41         std::string t = trim(line);
42         if (!t.empty()) v.push_back(t);
43     }
44     return v;
45 }
46
47 // проверка формата графа
48 bool validate_pairs_graph(const std::string& input, std::string& error)
49 {
50     auto lines = split_lines(input);
51     if (lines.size() < 7) { error = "Минимум 6 строк графа + start end
52         ."; return false; }
```

```

51
52     int N = lines.size() - 1;           // количество вершин = строк смежнос
ти
53     if (N < 6 || N > 20) {
54         error = "Число вершин должно быть 6..20.";
55         return false;
56     }
57
58     int start, end;           // последняя строка - начало и конец пути
59     {
60         std::istringstream s(lines.back());
61         if (!(s>>start>>end)) { error="Последняя строка: 'start end'";
            return false; }
62     }
63
64     if (start<0 || start>=N || end<0 || end>=N) {
65         error = "start/end вне диапазона.";
66         return false;
67     }
68
69     int edge_count = 0;
70
71     // обработка каждой строки смежности
72     for (int i=0; i<N; i++) {
73         std::istringstream iss(lines[i]);
74         std::vector<int> tok;
75         int x;
76         while (iss>>x) tok.push_back(x);
77
78         if (tok.size() % 2 != 0) {
79             error = "Строка " + std::to_string(i) + " имеет нечётное чис
ло токенов.";
80             return false;
81         }
82
83         // проверка каждой пары
84         for (size_t k=0; k<tok.size(); k+=2) {
85             int v = tok[k]; // номер соседней вершины
86             int w = tok[k+1]; // вес ребра
87             if (v<0 || v>=N) { error="Сосед вне диапазона"; return false
; }
88             if (w<0) { error="Отрицательный вес недопустим"; return
false; }
89             if (v>i) edge_count++; //чтобы избежать дублей
90         }
91     }
92
93     if (edge_count < 6) {
94         error = "Должно быть >=6 рёбер.";
95         return false;
96     }
97
98     return true;
99 }
100
101 // tcp отправка

```

```

102 bool tcp_send(int sock, const std::string& data) {
103     uint32_t data_size = htonl(data.size());    // преобразование длины
           в сетевой порядок байтов
104     if (send(sock, &data_size, sizeof(data_size), 0) != sizeof(data_size)
           )) {
105         return false;
106     }
107
108     // отправка данных
109     ssize_t total_sent = 0;
110     while (total_sent < data.size()) {
111         ssize_t n = send(sock, data.c_str() + total_sent, data.size() -
           total_sent, 0);
112         if (n <= 0) {
113             return false;
114         }
115         total_sent += n;
116     }
117
118     return true;
119 }
120
121 // tcp получение результата
122 bool tcp_recv_result(int sock, std::string& result) {
123     uint32_t resp_size;
124     if (recv(sock, &resp_size, sizeof(resp_size), 0) != sizeof(resp_size)
           )) {
125         return false;
126     }
127     resp_size = ntohl(resp_size);    // возвращение из сетевого порядка
128
129     if (resp_size > 8192) {          //чтобы не было слишком длинных ответов
130         return false;
131     }
132
133     std::vector<char> buffer(resp_size);
134     ssize_t total_read = 0;
135
136     // чтение ответа
137     while (total_read < resp_size) {
138         ssize_t n = recv(sock, buffer.data() + total_read, resp_size -
           total_read, 0);
139         if (n <= 0) {
140             return false;
141         }
142         total_read += n;
143     }
144
145     if (total_read == resp_size) {
146         result.assign(buffer.begin(), buffer.end());
147         return true;
148     }
149
150     return false;
151 }
152

```

```

153 // udp отправка
154 bool udp_send(int sock,const sockaddr_in& srv,std::string data){
155     const int MAX=3;    // максимум попыток
156     const int T=3000;    // время на ожидание подтверждения
157     socklen_t len=sizeof(srv);
158
159     for(int attempt=1;attempt<=MAX;attempt++){
160         // отправка данных
161         sendto(sock,data.c_str(),data.size(),0,(sockaddr*)&srv,len);
162
163         // ожидание подтверждения
164         pollfd pfd{sock,POLLIN,0};
165         if (poll(&pfd,1,T)>0){
166             if (pfd.revents & POLLIN){
167                 char buf[64];
168                 sockaddr_in from;
169                 socklen_t l=sizeof(from);
170                 int n=recvfrom(sock,buf,sizeof(buf)-1,0,(sockaddr*)&from
171                     ,&l);
172                 if (n>0){
173                     buf[n]='\0';
174                     // проверка что подтверждение корректно
175                     if (std::string(buf)=="DATA_ACK" &&
176                         from.sin_addr.s_addr==srv.sin_addr.s_addr &&
177                         from.sin_port==srv.sin_port){
178                         return true;
179                     }
180                 }
181             }
182         }
183         return false;
184     }
185
186 // udp получение результата
187 bool udp_recv_result(int sock,const sockaddr_in& srv,std::string& result
188 ) {
189     const int T=10000;    // время на ожидание ответа
190     pollfd pfd{sock,POLLIN,0};
191     if (poll(&pfd,1,T)<=0) return false;
192
193     char buf[8192];
194     sockaddr_in from;
195     socklen_t l=sizeof(from);
196     int n=recvfrom(sock,buf,sizeof(buf)-1,0,(sockaddr*)&from,&l);
197     if(n<=0) return false;
198     buf[n]='\0';
199
200     // отправка подтверждения получения ответа
201     if(from.sin_addr.s_addr==srv.sin_addr.s_addr && from.sin_port==srv.
202         sin_port){
203         result=buf;
204         const char* ack="RESULT_ACK";
205         sendto(sock,ack,strlen(ack),0,(sockaddr*)&srv,sizeof(srv));
206         return true;
207     }

```



```

206     return false;
207 }
208
209 int main(int argc, char*argv[]){
210     // проверка аргументов командной строки: IP, PORT, протокол
211     if(argc!=4){
212         std::cerr<<"Использование: "<<argv[0]<<" <IP> <PORT> <TCP|UDP>\n";
213         return 1;
214     }
215
216     std::string ip=argv[1];
217     int port=std::stoi(argv[2]);
218     std::string proto=argv[3];
219     bool tcp = (proto=="TCP" || proto=="tcp");
220
221     // создание сокета
222     int sock=socket(AF_INET, tcp?SOCK_STREAM:SOCK_DGRAM,0);
223     if(sock<0){perror("socket");return 1;}
224
225     // настройка адреса сервера
226     sockaddr_in srv{};
227     srv.sin_family=AF_INET;
228     srv.sin_port=htons(port);
229     inet_pton(AF_INET, ip.c_str(), &srv.sin_addr);
230
231     // установка соединения/привязка порта
232     if(tcp){
233         if(connect(sock, (sockaddr*)&srv, sizeof(srv))<0){
234             perror("connect");return 1;
235         }
236         std::cout<<"TCP соединение установлено.\n";
237     } else {
238         sockaddr_in loc{};
239         loc.sin_family=AF_INET;
240         loc.sin_port=0;
241         loc.sin_addr.s_addr=INADDR_ANY;
242         bind(sock, (sockaddr*)&loc, sizeof(loc));
243         std::cout<<"UDP клиент готов.\n";
244     }
245
246     char choice;
247     std::cout << "Выберите способ ввода графа:\n";
248     std::cout << "1. Ввести вручную\n";
249     std::cout << "2. Загрузить из файла\n";
250     std::cout << "Ваш выбор (1/2): ";
251     std::cin >> choice;
252     std::cin.ignore();
253
254     std::vector<std::string> graph_lines;
255
256     while(true){
257         std::string input, line;
258
259         if (choice == '2') {
260             // чтение из файла

```

```

261     std::cout << "Введите путь к файлу: ";
262     std::string filepath;
263     std::getline(std::cin, filepath);
264
265     std::ifstream file(filepath);
266     if (!file.is_open()) {
267         std::cerr << "Ошибка: не удалось открыть файл " <<
268             filepath << "\n";
269         choice = '1'; // переключение на ручной ввод
270         continue;
271     }
272
273     // чтение из файла
274     std::string file_content;
275     while (std::getline(file, line)) {
276         if (trim(line) == "exit") {
277             std::cout << "Обнаружено 'exit' в файле. Клиент заве
278                 ршён.\n";
279             close(sock);
280             return 0;
281         }
282         file_content += line + "\n";
283     }
284     file.close();
285
286     if (file_content.empty()) {
287         std::cerr << "Ошибка: файл пуст или содержит только пуст
288             ые строки\n";
289         choice = '1'; // Переключаем на ручной ввод
290         continue;
291     }
292
293     // валидация входных данных
294     std::string err;
295     if(!validate_pairs_graph(file_content, err)) {
296         std::cerr << "Ошибка валидации данных из файла: " << err
297             << "\n";
298         choice = '1'; // Переключаем на ручной ввод
299         continue;
300     }
301
302     input = file_content;
303     std::cout << "Граф успешно загружен из файла.\n";
304
305     choice = '1';
306
307 } else {
308     // ручной ввод
309     std::cout << "Введите граф (векторы смежности). Пустая строк
310         а - отправка. exit - выход.\n";
311
312     while(true){
313         std::cout<<"> ";
314         if(!std::getline(std::cin, line)){
315             break;
316         }
317     }

```

```

312         std::string t=trim(line);
313         if(t=="exit"){
314             std::cout<<"Клиент завершён.\n";
315             close(sock);
316             return 0;
317         }
318         if(t.empty()){
319             break;
320         }
321         input+=line+"\n";
322     }
323
324     if(input.empty()) continue;
325
326     // валидация входных данных
327     std::string err;
328     if(!validate_pairs_graph(input, err)){
329         std::cerr<<"Ошибка: "<<err<<"\n";
330         continue;
331     }
332 }
333
334 // отправка данных на сервер и получение результата
335 if(tcp){
336     if (!tcp_send(sock, input)) {
337         std::cerr << "Ошибка отправки данных\n";
338         continue;
339     }
340
341     std::string result;
342     if (!tcp_rcv_result(sock, result)) {
343         std::cerr << "Ошибка получения результата\n";
344         continue;
345     }
346
347     std::cout << result;
348 } else {
349     if(!udp_send(sock, srv, input)){
350         std::cerr<<"Не получил DATA_ACK\n";
351         continue;
352     }
353     std::string result;
354     if(!udp_rcv_result(sock, srv, result)){
355         std::cerr<<"Не получил результат.\n";
356         continue;
357     }
358     std::cout<<result;
359 }
360
361 std::cout << "\nВыберите способ ввода графа:\n";
362 std::cout << "1. Ввести вручную\n";
363 std::cout << "2. Загрузить из файла\n";
364 std::cout << "Ваш выбор (1/2): ";
365 std::cin >> choice;
366 std::cin.ignore();
367 }

```

```
368
369     close(sock);
370     return 0;
371 }
```

Приложение Б. Исходный код серверной части

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5 #include <cstring>
6 #include <sstream>
7 #include <thread>
8 #include <mutex>
9 #include <signal.h>
10 #include <atomic>
11 #include <unistd.h>
12 #include <sys/socket.h>
13 #include <arpa/inet.h>
14 #include <poll.h>
15 #include <climits>
16
17 std::atomic<bool> running(true);
18 std::mutex log_m;
19
20 // логирование
21 void log(const std::string&s){
22     std::lock_guard<std::mutex>k(log_m);
23     std::cout<<"[SERVER] " <<s<<std::endl;
24 }
25
26 // удаление пробелов
27 static inline std::string trim(const std::string&s){
28     size_t a=s.find_first_not_of(" \t\r\n");
29     if(a==std::string::npos)return "";
30     size_t b=s.find_last_not_of(" \t\r\n");
31     return s.substr(a,b-a+1);
32 }
33
34 // разделение текста на строки
35 std::vector<std::string> split(const std::string&txt){
36     std::vector<std::string>v;
37     std::istringstream iss(txt);
38     std::string l;
39     while(std::getline(iss,l)){
40         auto t=trim(l);
41         if(!t.empty())v.push_back(t);
42     }
43     return v;
44 }
45
46 // парсинг входных данных графа
47 bool parse_pairs_graph(
48     const std::string&input,
49     std::vector<std::vector<std::pair<int,int>>>&adj,
50     int&start,int&end,
51     std::string&err
52 ){
```

```

53 auto lines=split(input);
54 if(lines.size()<7){err="Минимум 6 вершин + старт-энд.";return false
    ;}
55
56 int N=lines.size()-1;
57 if(N<6||N>20){err="Число вершин 6..20.";return false;}
58
59 // обработка последней строки - начало и конец пути
60 {
61     std::istringstream se(lines.back());
62     if(!(se>>start>>end)){err="Последняя строка: start end";return
        false;}
63     if(start<0||start>=N||end<0||end>=N){err="start/end вне диапазон
        а.";return false;}
64 }
65
66 adj.assign(N,{});
67 int edges=0;
68
69 // обработка каждой строки
70 for(int i=0;i<N;i++){
71     std::istringstream iss(lines[i]);
72     std::vector<int>tok;
73     int x;while(iss>>x)tok.push_back(x);
74
75     // токенов должно быть четное кол-во, потому что это пары: сосед
76     // ная вершина + вес
77     if(tok.size()%2!=0){err="Нечётное число токенов в строке "+std::
78         to_string(i);return false;}
79
80     for(size_t k=0;k<tok.size();k+=2){
81         int v=tok[k],w=tok[k+1];
82         if(v<0||v>=N){err="Сосед вне диапазона.";return false;}
83         if(w<0){err="Отрицательный вес.";return false;}
84         adj[i].push_back({v,w});
85         if(v>i)edges++; // чтобы избежать дубликатов
86     }
87 }
88
89
90 // проверка связности
91 bool connected(const std::vector<std::vector<std::pair<int,int>>>&a){
92     int N=a.size();
93     std::vector<bool>v(N,false);
94     std::queue<int>q;
95     q.push(0);v[0]=true;
96     int c=1;
97     while(!q.empty()){
98         int u=q.front();q.pop();
99         for(auto&p:a[u]){
100             int x=p.first;
101             if(!v[x]){v[x]=true;c++;q.push(x);}
102         }
103     }

```

```

104     return c==N;
105 }
106
107 // алгоритм дейкстры
108 std::pair<int, std::vector<int>> dijkstra(
109     const std::vector<std::vector<std::pair<int, int>>>&adj,
110     int s, int e
111 ){
112     int N=adj.size();
113     // расстояния, родительские вершины (чтобы восстановить путь потом),
114     // посещенные вершины
115     std::vector<int>d(N, INT_MAX), p(N, -1); std::vector<bool>u(N, false);
116     d[s]=0;
117
118     for(int i=0; i<N; i++){
119         int x=-1;
120         for(int j=0; j<N; j++){
121             if(!u[j] && (x==-1 || d[j]<d[x])) x=j;
122             if(x==-1 || d[x]==INT_MAX) break;
123             u[x]=true;
124             for(auto&pp: adj[x]){
125                 int v=pp.first, w=pp.second;
126                 if(d[x]+w<d[v]){
127                     d[v]=d[x]+w; p[v]=x;
128                 }
129             }
130         }
131
132         if(d[e]==INT_MAX) return{-1, {}}; // конечная вершина недостижима
133         std::vector<int>path;
134         for(int v=e; v!=-1; v=p[v]) path.push_back(v); // восстановление пу
135         // ти
136         std::reverse(path.begin(), path.end());
137         return{d[e], path};
138     }
139
140 // формирование ответа
141 std::string response(
142     bool ok, const std::string&err,
143     const std::vector<std::vector<std::pair<int, int>>>&adj,
144     int s, int e
145 ){
146     if(!ok) return "Ошибка: "+err+"\n";
147     if(!connected(adj)) return "Ошибка: граф несвязный!\n";
148     auto[r, path]=dijkstra(adj, s, e);
149     if(r<0) return "Ошибка: путь не существует!\n";
150
151     std::ostringstream out;
152     out<<"Минимальный путь = "<<r<<"\nПуть: ";
153     for(size_t i=0; i<path.size(); i++){
154         if(i>0) out<<" -> ";
155         out<<path[i];
156     }
157     out<<"\n";
158     return out.str();
159 }

```

```

158
159 // udp отправка результата
160 bool udp_send_result(int sock,const std::string& msg, sockaddr_in client
    ,socklen_t len){
161     const int MAX=3;    // кол-во попыток
162     const int T=3000;   // время ожидания
163     for(int a=1;a<=MAX;a++){
164         // отправка сообщения
165         sendto(sock,msg.c_str(),msg.size(),0,(sockaddr*)&client,len);
166
167         // ожидание подтверждения
168         pollfd pfd{sock,POLLIN,0};
169         if(poll(&pfd,1,T)>0){
170             char buf[64];
171             sockaddr_in from; socklen_t fl=sizeof(from);
172             int n=recvfrom(sock,buf,sizeof(buf)-1,0,(sockaddr*)&from,&fl
                );
173             if(n>0){
174                 buf[n]='\0';
175                 // проверка корректности подтверждения
176                 if(std::string(buf)=="RESULT_ACK" &&
177                     from.sin_addr.s_addr==client.sin_addr.s_addr &&
178                     from.sin_port==client.sin_port){
179                     return true;
180                 }
181             }
182         }
183     }
184     return false;
185 }
186
187 // udp обработка
188 void udp_handle(std::string data, sockaddr_in client, socklen_t len, int
    sock){
189     // отправка подтверждения получения данных
190     const char* ack="DATA_ACK";
191     sendto(sock,ack,strlen(ack),0,(sockaddr*)&client,len);
192
193     std::vector<std::vector<std::pair<int,int>>> adj;
194     int s,e;
195     std::string err;
196     bool ok=parse_pairs_graph(data,adj,s,e,err);
197
198     std::string resp=response(ok,err,adj,s,e);
199     udp_send_result(sock,resp,client,len);
200 }
201
202 // tcp обработка
203 void tcp_handle(int cs, std::string info){
204     // получения сообщения
205     uint32_t data_size;
206     if (recv(cs, &data_size, sizeof(data_size), 0) != sizeof(data_size))
207     {
208         close(cs);
209         return;
210     }

```



```

210     data_size = ntohl(data_size);
211
212     if (data_size > 8192) {          // чтобы не было слишком длинных сообщен
        ий
213         close(cs);
214         return;
215     }
216
217     std::vector<char> buffer(data_size);
218     ssize_t total_read = 0;
219
220     // чтение сообщения
221     while (total_read < data_size) {
222         ssize_t n = recv(cs, buffer.data() + total_read, data_size -
            total_read, 0);
223         if (n <= 0) {
224             close(cs);
225             return;
226         }
227         total_read += n;
228     }
229
230     std::string d(buffer.begin(), buffer.end());
231
232     std::vector<std::vector<std::pair<int,int>>> adj;
233     int s, e;
234     std::string err;
235     bool ok = parse_pairs_graph(d, adj, s, e, err);
236     std::string resp = response(ok, err, adj, s, e);
237
238     // отправка ответа
239     uint32_t resp_size = htonl(resp.size());
240     send(cs, &resp_size, sizeof(resp_size), 0);
241
242     send(cs, resp.c_str(), resp.size(), 0);
243     close(cs);
244 }
245
246 // обработка Ctrl+C
247 void handler(int){
248     running=false;
249     log("Получен SIGINT - завершение...");
250 }
251
252 int main(int argc, char*argv[]){
253     if(argc!=2){
254         std::cerr<<"Использование: "<<argv[0]<<" <порт>\n";return 1;
255     }
256     int port=std::stoi(argv[1]);
257     signal(SIGINT, handler);
258
259     // создание сокетов
260     int tcp=socket(AF_INET, SOCK_STREAM, 0);
261     int udp=socket(AF_INET, SOCK_DGRAM, 0);
262
263     int opt=1;

```

```

264     setsockopt(tcp, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
265     setsockopt(udp, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
266
267     // настройка адреса сервера
268     sockaddr_in addr{};
269     addr.sin_family=AF_INET;
270     addr.sin_port=htons(port);
271     addr.sin_addr.s_addr=INADDR_ANY;
272
273     // привязка сокетов
274     bind(tcp, (sockaddr*)&addr, sizeof(addr));
275     bind(udp, (sockaddr*)&addr, sizeof(addr));
276
277     // запуск прослушивания TCP-соединений
278     listen(tcp, 10);
279     log("Сервер запущен");
280
281     while(running){
282         fd_set fds;
283         FD_ZERO(&fds);
284         FD_SET(tcp, &fds);
285         FD_SET(udp, &fds);
286
287         timeval tv{1, 0};
288         int r=select(std::max(tcp, udp)+1, &fds, nullptr, nullptr, &tv);
289         if(r<=0) continue;
290         if(!running) break;
291
292         // обработка нового TCP-соединения
293         if(FD_ISSET(tcp, &fds)){
294             sockaddr_in cl; socklen_t l=sizeof(cl);
295             int cs=accept(tcp, (sockaddr*)&cl, &l);
296             if(cs>=0){
297                 std::string info=inet_ntoa(cl.sin_addr);
298                 info+=":" + std::to_string(ntohs(cl.sin_port));
299                 std::thread(tcp_handle, cs, info).detach(); // обраб
300                     отка в отдельном потоке
301             }
302         }
303
304         // обработка нового UDP-соединения
305         if(FD_ISSET(udp, &fds)){
306             char buf[8192];
307             socklen_t l=sizeof(cl);
308             int n=recvfrom(udp, buf, sizeof(buf)-1, 0, (sockaddr*)&cl, &l);
309             if(n>0){
310                 buf[n]='\0';
311                 std::thread(udp_handle, std::string(buf), cl, l, udp).detach
312                     (); // обработка в отдельном потоке
313             }
314         }
315     }
316
317     close(tcp);
318     close(udp);
319     log("Сервер завершён.");

```


Приложение В. Исходный код тестирующего приложения (bash-скрипт)

```
1 #!/bin/bash
2
3 set -e
4
5 PORT=${1:-12345}
6 IP=127.0.0.1
7
8 for bin in ./server ./client ./expect_graph_test.exp; do
9     if [ ! -e "$bin" ]; then
10         echo "Ошибка: файл '$bin' не найден."
11         exit 1
12     fi
13     chmod +x "$bin"
14 done
15
16 echo "Запускаю сервер на порту $PORT ..."
17 ./server "$PORT" >/tmp/server_test.log 2>&1 &
18 SERVER_PID=$!
19 sleep 1
20
21 if ! kill -0 "$SERVER_PID" 2>/dev/null; then
22     echo "Ошибка: сервер не запустился."
23     exit 1
24 fi
25
26 echo "Сервер запущен (PID: $SERVER_PID)"
27
28 tests=("ok" "negweight" "disconnected" "start_out" "not_enough_edges")
29 failed=0
30
31 run_test() {
32     local test_name=$1
33     local proto=$2
34     echo "=== Тест: $test_name ($proto) ==="
35     if ! expect ./expect_graph_test.exp "$IP" "$PORT" "$proto" "
36         $test_name"; then
37         echo "Тест $test_name ($proto) завершился с ошибкой"
38         ((failed++))
39     fi
40 }
41
42 for t in "${tests[@]}"; do
43     run_test "$t" "TCP"
44 done
45
46 for t in "${tests[@]}"; do
47     run_test "$t" "UDP"
48 done
49
50 echo
51 echo "Останавливаю сервер (PID $SERVER_PID)..."
```

```
51 kill "$SERVER_PID" 2>/dev/null || true
52 wait "$SERVER_PID" 2>/dev/null || true
53
54 echo "Логи сервера сохранены в /tmp/server_test.log"
55
56 if [ $failed -eq 0 ]; then
57     echo -e "\nВсе тесты пройдены успешно!"
58     exit 0
59 else
60     echo -e "\n$failed тест(ов) завершились с ошибкой."
61     exit 1
62 fi
```

Приложение Г. Исходный код тестирующего приложения (Ехрест-скрипт)

```
1 #!/usr/bin/expect -f
2
3 set timeout 15
4 set ip [lindex $argv 0]
5 set port [lindex $argv 1]
6 set proto [lindex $argv 2]
7 set test [lindex $argv 3]
8
9 set valid_tests [list "ok" "negweight" "disconnected" "start_out" "
   not_enough_edges" "n5" "n6" "n20" "n21" "file_ok"]
10 if {[lsearch -exact $valid_tests $test] == -1} {
11     puts "Unknown test: $test"
12     exit 2
13 }
14
15 spawn ./client $ip $port $proto
16
17 expect {
18     -re "TCP соединение установлено\\\\" {}
19     -re "UDP клиент готов\\\\" {}
20     timeout { puts "FAILED: client didn't print ready message"; exit 2 }
21 }
22
23 expect {
24     -re {Ваш выбор.*1/2.*:} {
25         send -- "1\r"
26     }
27     timeout { puts "FAILED: не получил меню выбора способа ввода"; exit
28         2 }
29 }
30 expect {
31     -re "Введите граф.*" {}
32     timeout { puts "FAILED: client didn't print input prompt"; exit 2 }
33 }
34
35 expect -re {> }
36
37 # Define test cases
38 if {$test == "ok"} {
39     set lines {
40         "1 1 2 2"
41         "0 1 2 3"
42         "0 2 3 1"
43         "0 1 4 1"
44         "1 2 5 2"
45         "2 1"
46         "0 5"
47     }
48     set expect_re {Минимальный путь =}
49 } elseif {$test == "negweight" } {
```

```

50     set lines {
51         "1 -1"
52         "0 1"
53         "0 1"
54         "0 1"
55         "0 1"
56         "0 1"
57         "0 5"
58     }
59     set expect_re {Ошибка:}
60 } elseif {$test == "disconnected"} {
61     set lines {
62         "1 1 2 1"
63         "0 1"
64         ""
65         "3 1 4 1"
66         ""
67         ""
68         "0 5"
69     }
70     set expect_re {Ошибка: граф несвязный!|Ошибка:}
71 } elseif {$test == "start_out"} {
72     set lines {
73         "1 1 2 2"
74         "0 1 2 3"
75         "0 2 3 1"
76         "0 1 4 1"
77         "1 2 5 2"
78         "2 1"
79         "10 0"
80     }
81     set expect_re {Ошибка:}
82 } elseif {$test == "not_enough_edges"} {
83     set lines {
84         "1 1"
85         "0 1"
86         "0 1"
87         "0 1"
88         "0 1"
89         "0 1"
90         "0 5"
91     }
92     set expect_re {Ошибка:}
93 } elseif {$test == "n5"} {
94     set lines {
95         "1 1"
96         "0 1"
97         "0 1"
98         "0 1"
99         "0 1"
100        "0 4"
101    }
102    set expect_re {Ошибка:.*Число вершин 6\\.\\.20\\.}
103 } elseif {$test == "n6"} {
104     set lines {
105         "1 1 2 2 3 1 4 1 5 1" ;# 0 -> 1,2,3,4,5

```

```

106         "0 1"                ;# 1 -> 0
107         "0 1"                ;# 2 -> 0
108         "0 1"                ;# 3 -> 0
109         "0 1"                ;# 4 -> 0
110         "0 1"                ;# 5 -> 0
111         "0 5"
112     }
113     set expect_re {Минимальный путь =}
114 } elseif {$test == "n20"} {
115     set lines {}
116     set line0 ""
117     for {set i 1} {$i <= 19} {incr i} {
118         append line0 "$i 1 "
119     }
120     lappend lines [string trim $line0]
121     for {set i 1} {$i <= 19} {incr i} {
122         lappend lines "0 1"
123     }
124     lappend lines "0 19"
125     set expect_re {Минимальный путь =}
126 } elseif {$test == "n21"} {
127     set lines {}
128     for {set i 0} {$i < 21} {incr i} {
129         lappend lines "0 1" ;
130     }
131     lappend lines "0 20" ;
132     set expect_re {Ошибка:.*Число вершин 6\\.\\.20\\.}
133 } elseif {$test == "file_ok"} {
134     set expect_re {Минимальный путь =}
135     set filepath "test_graphs/file_ok.txt"
136 }
137
138 if {$test == "file_ok"} {
139     expect {
140         -re {Ваш выбор.*1/2.*:} {
141             send -- "2\r"
142         }
143         timeout { puts "FAILED: не получил меню выбора"; exit 2 }
144     }
145
146     expect {
147         -re {Введите путь к файлу:} {
148             send -- "$filepath\r"
149         }
150         timeout { puts "FAILED: не запросил путь к файлу"; exit 2 }
151     }
152 } else {
153     expect -re {> }
154     foreach l $lines {
155         send -- "$l\r"
156         expect {
157             -re {> } {}
158             timeout { }
159         }
160     }
161     send -- "\r"

```



```

162 }
163
164 send -- "\r"
165
166 expect {
167     -re $expect_re {
168         puts "OK: got expected response for test $test"
169
170         expect {
171             -re {Ваш выбор.*1/2.*:} {
172                 send -- "1\r"
173             }
174             timeout {
175                 send -- "exit\r"
176                 expect eof
177                 exit 0
178             }
179         }
180
181         expect {
182             -re {> } {
183                 send -- "exit\r"
184             }
185             timeout {
186                 send -- "exit\r"
187             }
188         }
189
190         expect eof
191         exit 0
192     }
193     timeout {
194         puts "FAILED: timeout waiting for server response (test $test)"
195         send -- "\x03" ;# Ctrl+C
196         expect eof
197         exit 3
198     }
199 }

```

Приложение Д. Протоколы тестирования

```
1 Запускаю сервер на порту 12345 ...
2 Сервер запущен (PID: 3285)
3 === Тест: ok (TCP) ===
4 spawn ./client 127.0.0.1 12345 TCP
5 TCP соединение установлено.
6 Выберите способ ввода графа:
7 1. Ввести вручную
8 2. Загрузить из файла
9 Ваш выбор (1/2): 1
10 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
11 .
12 > 1 1 2 2
13 > 0 1 2 3
14 > 0 2 3 1
15 > 0 1 4 1
16 > 1 2 5 2
17 > 2 1
18 > 0 5
19 >
20 Минимальный путь = 6
21 Путь: 0 -> 2 -> 3 -> 4 -> 5
22 Выберите способ ввода графа:
23 1. Ввести вручную
24 2. Загрузить из файла
25 Ваш выбор (1/2): OK: got expected response for test ok
26 1
27 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
28 .
29 > exit
30 Клиент завершён.
31 === Тест: negweight (TCP) ===
32 spawn ./client 127.0.0.1 12345 TCP
33 TCP соединение установлено.
34 Выберите способ ввода графа:
35 1. Ввести вручную
36 2. Загрузить из файла
37 Ваш выбор (1/2): 1
38 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
39 .
40 > 1 -1
41 > 0 1
42 > 0 1
43 > 0 1
44 > 0 1
45 > 0 1
46 > 0 1
47 > 0 5
48 >
49 Ошибка: Отрицательный вес недопустим
50 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
51 .
52 > OK: got expected response for test negweight
53 exit
54 Клиент завершён.
```

```

51 === Тест: disconnected (TCP) ===
52 spawn ./client 127.0.0.1 12345 TCP
53 TCP соединение установлено.
54 Выберите способ ввода графа:
55 1. Ввести вручную
56 2. Загрузить из файла
57 Ваш выбор (1/2): 1
58 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
59 .
60 > 1 1 2 1
61 > 0 1
62 >
63 Ошибка: Минимум 6 строк графа + start end.
64 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
65 .
66 > 3 1 4 1
67 >
68 Ошибка: Минимум 6 строк графа + start end.
69 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
70 .
71 > 0 5
72 >
73 Ошибка: Минимум 6 строк графа + start end.
74 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
75 .
76 > OK: got expected response for test disconnected
77 exit
78 Клиент завершён.
79 === Тест: start_out (TCP) ===
80 spawn ./client 127.0.0.1 12345 TCP
81 TCP соединение установлено.
82 Выберите способ ввода графа:
83 1. Ввести вручную
84 2. Загрузить из файла
85 Ваш выбор (1/2): 1
86 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
87 .
88 > 1 1 2 2
89 > 0 1 2 3
90 > 0 2 3 1
91 > 0 1 4 1
92 > 1 2 5 2
93 > 2 1
94 > 10 0
95 >
96 Ошибка: start/end вне диапазона.
97 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
98 .
99 > OK: got expected response for test start_out
100 exit
101 Клиент завершён.
102 === Тест: not_enough_edges (TCP) ===
103 spawn ./client 127.0.0.1 12345 TCP

```

```

100 TCP соединение установлено.
101 Выберите способ ввода графа:
102 1. Ввести вручную
103 2. Загрузить из файла
104 Ваш выбор (1/2): 1
105 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
106 .
107 > 1 1
108 > 0 1
109 > 0 1
110 > 0 1
111 > 0 1
112 > 0 1
113 >
114 Ошибка: Должно быть >=6 рёбер.
115 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
116 .
117 > OK: got expected response for test not_enough_edges
118 exit
119 Клиент завершён.
120 === Тест: n5 (TCP) ===
121 spawn ./client 127.0.0.1 12345 TCP
122 TCP соединение установлено.
123 Выберите способ ввода графа:
124 1. Ввести вручную
125 2. Загрузить из файла
126 Ваш выбор (1/2): 1
127 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
128 .
129 > 1 1
130 > 0 1
131 > 0 1
132 > 0 1
133 > 0 1
134 > 0 1
135 > 0 4
136 >
137 Ошибка: Число вершин должно быть 6..20.
138 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
139 .
140 > OK: got expected response for test n5
141 exit
142 Клиент завершён.
143 === Тест: n6 (TCP) ===
144 spawn ./client 127.0.0.1 12345 TCP
145 TCP соединение установлено.
146 Выберите способ ввода графа:
147 1. Ввести вручную
148 2. Загрузить из файла
149 Ваш выбор (1/2): 1
150 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
151 .
152 > 1 1 2 2 3 1 4 1 5 1
153 > 0 1
154 > 0 1
155 > 0 1

```

```

151 > 0 1
152 > 0 1
153 > 0 5
154 >
155 Минимальный путь = 2
156 Путь: 0 -> 5
157
158 Выберите способ ввода графа:
159 1. Ввести вручную
160 2. Загрузить из файла
161 Ваш выбор (1/2): OK: got expected response for test n6
162 1
163 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
164 .
165 > exit
166 Клиент завершён.
167 === Тест: n20 (TCP) ===
168 spawn ./client 127.0.0.1 12345 TCP
169 TCP соединение установлено.
170 Выберите способ ввода графа:
171 1. Ввести вручную
172 2. Загрузить из файла
173 Ваш выбор (1/2): 1
174 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
175 .
176 > 1 1 2 1 3 1 4 1 5 1 6 1 7 1 8 1 9 1 10 1 11 1 12 1 13 1 14 1 15 1 16 1
177 17 1 18 1 19 1
178 > 0 1
179 > 0 1
180 > 0 1
181 > 0 1
182 > 0 1
183 > 0 1
184 > 0 1
185 > 0 1
186 > 0 1
187 > 0 1
188 > 0 1
189 > 0 1
190 > 0 1
191 > 0 1
192 > 0 1
193 > 0 19
194 >
195 Минимальный путь = 2
196 Путь: 0 -> 19
197
198 Выберите способ ввода графа:
199 1. Ввести вручную
200 2. Загрузить из файла
201 Ваш выбор (1/2): OK: got expected response for test n20
202 1
203 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход

```

```

.
204 > exit
205 Клиент завершён.
206 === Тест: n21 (TCP) ===
207 spawn ./client 127.0.0.1 12345 TCP
208 TCP соединение установлено.
209 Выберите способ ввода графа:
210 1. Ввести вручную
211 2. Загрузить из файла
212 Ваш выбор (1/2): 1
213 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
.
214 > 0 1
215 > 0 1
216 > 0 1
217 > 0 1
218 > 0 1
219 > 0 1
220 > 0 1
221 > 0 1
222 > 0 1
223 > 0 1
224 > 0 1
225 > 0 1
226 > 0 1
227 > 0 1
228 > 0 1
229 > 0 1
230 > 0 1
231 > 0 1
232 > 0 1
233 > 0 1
234 > 0 1
235 > 0 20
236 >
237 Ошибка: Число вершин должно быть 6..20.
238 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
.
239 > OK: got expected response for test n21
240 exit
241 Клиент завершён.
242 === Тест: file_ok (TCP) ===
243 spawn ./client 127.0.0.1 12345 TCP
244 TCP соединение установлено.
245 Выберите способ ввода графа:
246 1. Ввести вручную
247 2. Загрузить из файла
248 Ваш выбор (1/2): 2
249 Введите путь к файлу: test_graphs/file_ok.txt
250 Граф успешно загружен из файла.
251 Минимальный путь = 6
252 Путь: 0 -> 2 -> 3 -> 4 -> 5
253
254 Выберите способ ввода графа:
255 1. Ввести вручную
256 2. Загрузить из файла

```

```

257 Ваш выбор (1/2): OK: got expected response for test file_ok
258 1
259 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
260 .
261 > exit
262 Клиент завершён.
263 === Тест: ok (UDP) ===
264 spawn ./client 127.0.0.1 12345 UDP
265 UDP клиент готов.
266 Выберите способ ввода графа:
267 1. Ввести вручную
268 2. Загрузить из файла
269 Ваш выбор (1/2): 1
270 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
271 .
272 > 1 1 2 2
273 > 0 1 2 3
274 > 0 2 3 1
275 > 0 1 4 1
276 > 1 2 5 2
277 > 2 1
278 > 0 5
279 >
280 Минимальный путь = 6
281 Путь: 0 -> 2 -> 3 -> 4 -> 5
282
283 Выберите способ ввода графа:
284 1. Ввести вручную
285 2. Загрузить из файла
286 Ваш выбор (1/2): OK: got expected response for test ok
287 1
288 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
289 .
290 > exit
291 Клиент завершён.
292 === Тест: negweight (UDP) ===
293 spawn ./client 127.0.0.1 12345 UDP
294 UDP клиент готов.
295 Выберите способ ввода графа:
296 1. Ввести вручную
297 2. Загрузить из файла
298 Ваш выбор (1/2): 1
299 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
300 .
301 > 1 -1
302 > 0 1
303 > 0 1
304 > 0 1
305 > 0 1
306 > 0 1
307 > 0 1
308 > 0 5
309 >
310 Ошибка: Отрицательный вес недопустим
311 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
312 .
313 > OK: got expected response for test negweight

```

```

308 exit
309 Клиент завершён.
310 === Тест: disconnected (UDP) ===
311 spawn ./client 127.0.0.1 12345 UDP
312 UDP клиент готов.
313 Выберите способ ввода графа:
314 1. Ввести вручную
315 2. Загрузить из файла
316 Ваш выбор (1/2): 1
317 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
318 .
319 > 1 1 2 1
320 > 0 1
321 >
322 Ошибка: Минимум 6 строк графа + start end.
323 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
324 .
325 > 3 1 4 1
326 >
327 Ошибка: Минимум 6 строк графа + start end.
328 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
329 .
330 > 0 5
331 >
332 Ошибка: Минимум 6 строк графа + start end.
333 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
334 .
335 > OK: got expected response for test disconnected
336 exit
337 Клиент завершён.
338 === Тест: start_out (UDP) ===
339 spawn ./client 127.0.0.1 12345 UDP
340 UDP клиент готов.
341 Выберите способ ввода графа:
342 1. Ввести вручную
343 2. Загрузить из файла
344 Ваш выбор (1/2): 1
345 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
346 .
347 > 1 1 2 2
348 > 0 1 2 3
349 > 0 2 3 1
350 > 0 1 4 1
351 > 1 2 5 2
352 > 2 1
353 > 10 0
354 >
355 Ошибка: start/end вне диапазона.
356 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
357 .
358 > OK: got expected response for test start_out
359 exit
360 Клиент завершён.

```



```

357 === Тест: not_enough_edges (UDP) ===
358 spawn ./client 127.0.0.1 12345 UDP
359 UDP клиент готов.
360 Выберите способ ввода графа:
361 1. Ввести вручную
362 2. Загрузить из файла
363 Ваш выбор (1/2): 1
364 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
365 .
366 > 1 1
367 > 0 1
368 > 0 1
369 > 0 1
370 > 0 1
371 > 0 5
372 >
373 Ошибка: Должно быть >=6 рёбер.
374 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
375 .
376 > OK: got expected response for test not_enough_edges
377 exit
378 Клиент завершён.
379 === Тест: n5 (UDP) ===
380 spawn ./client 127.0.0.1 12345 UDP
381 UDP клиент готов.
382 Выберите способ ввода графа:
383 1. Ввести вручную
384 2. Загрузить из файла
385 Ваш выбор (1/2): 1
386 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
387 .
388 > 1 1
389 > 0 1
390 > 0 1
391 > 0 1
392 > 0 1
393 > 0 4
394 >
395 Ошибка: Число вершин должно быть 6..20.
396 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
397 .
398 > OK: got expected response for test n5
399 exit
400 Клиент завершён.
401 === Тест: n6 (UDP) ===
402 spawn ./client 127.0.0.1 12345 UDP
403 UDP клиент готов.
404 Выберите способ ввода графа:
405 1. Ввести вручную
406 2. Загрузить из файла
407 Ваш выбор (1/2): 1
408 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
409 .
410 > 1 1 2 2 3 1 4 1 5 1
411 > 0 1

```

```

408 > 0 1
409 > 0 1
410 > 0 1
411 > 0 1
412 > 0 1
413 > 0 5
414 >
415 Минимальный путь = 2
416 Путь: 0 -> 5
417
418 Выберите способ ввода графа:
419 1. Ввести вручную
420 2. Загрузить из файла
421 Ваш выбор (1/2): OK: got expected response for test n6
422 1
423 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
424 .
425 > exit
426 Клиент завершён.
427 === Тест: n20 (UDP) ===
428 spawn ./client 127.0.0.1 12345 UDP
429 UDP клиент готов.
430 Выберите способ ввода графа:
431 1. Ввести вручную
432 2. Загрузить из файла
433 Ваш выбор (1/2): 1
434 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
435 .
436 > 1 1 2 1 3 1 4 1 5 1 6 1 7 1 8 1 9 1 10 1 11 1 12 1 13 1 14 1 15 1 16 1
437 17 1 18 1 19 1
438 > 0 1
439 > 0 1
440 > 0 1
441 > 0 1
442 > 0 1
443 > 0 1
444 > 0 1
445 > 0 1
446 > 0 1
447 > 0 1
448 > 0 1
449 > 0 1
450 > 0 1
451 > 0 1
452 > 0 1
453 > 0 1
454 > 0 1
455 > 0 19
456 >
457 Минимальный путь = 2
458 Путь: 0 -> 19
459
460 Выберите способ ввода графа:

```

```

461 1. Ввести вручную
462 2. Загрузить из файла
463 Ваш выбор (1/2): 0K: got expected response for test n20
464 1
465 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
466 .
467 > exit
468 Клиент завершён.
469 === Тест: n21 (UDP) ===
470 spawn ./client 127.0.0.1 12345 UDP
471 UDP клиент готов.
472 Выберите способ ввода графа:
473 1. Ввести вручную
474 2. Загрузить из файла
475 Ваш выбор (1/2): 1
476 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
477 .
478 > 0 1
479 > 0 1
480 > 0 1
481 > 0 1
482 > 0 1
483 > 0 1
484 > 0 1
485 > 0 1
486 > 0 1
487 > 0 1
488 > 0 1
489 > 0 1
490 > 0 1
491 > 0 1
492 > 0 1
493 > 0 1
494 > 0 1
495 > 0 1
496 > 0 1
497 > 0 20
498 >
499 Ошибка: Число вершин должно быть 6..20.
500 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
501 .
502 > 0K: got expected response for test n21
503 exit
504 Клиент завершён.
505 === Тест: file_ok (UDP) ===
506 spawn ./client 127.0.0.1 12345 UDP
507 UDP клиент готов.
508 Выберите способ ввода графа:
509 1. Ввести вручную
510 2. Загрузить из файла
511 Ваш выбор (1/2): 2
512 Введите путь к файлу: test_graphs/file_ok.txt
513 Граф успешно загружен из файла.
514 Минимальный путь = 6

```

```
514 Путь: 0 -> 2 -> 3 -> 4 -> 5
515
516 Выберите способ ввода графа:
517 1. Ввести вручную
518 2. Загрузить из файла
519 Ваш выбор (1/2): 0K: got expected response for test file_ok
520 1
521 Введите граф (векторы смежности). Пустая строка - отправка. exit - выход
522 .
523 > exit
524 Клиент завершён.
525
526 Останавливаю сервер (PID 3285)...
527 Логи сервера сохранены в server_test.log
528
529 Все тесты пройдены успешно!
```